



Faculdade de Informática e Administração Paulista

Arthur Vettorazzo de Souza – RM 569445

Brayan Barbosa Dos Santos - RM 573682

Giovanne Gomes Petenuci - RM 574091

Global Solution 2026 - 1º Semestre

Projeto E.D.E.N. - Ecological Development in Exo-Environments

RELATÓRIO TÉCNICO

1CCPZ (CIÊNCIAS DA COMPUTAÇÃO - NOTURNO)

**Computer Organization and
Architecture**

Professor: Marcus Grilo

São Paulo

2026

Arthur Vettorazzo de Souza – RM 569445

Brayan Barbosa Dos Santos - RM 573682

Giovanne Gomes Petenuci - RM 574091

Global Solution 2026 - 1º Semestre

Projeto E.D.E.N. - Ecological Development in Exo-Environments

RELATÓRIO TÉCNICO

1CCPZ (CIÊNCIAS DA COMPUTAÇÃO - NOTURNO)

Trabalho apresentado à Faculdade
de Informática e Administração
Paulista - FIAP como parte das
exigências da disciplina de
**Computer Organization and
Architecture**

Professor: Marcus Grilo

São Paulo

2026

SUMÁRIO

1. INTRODUÇÃO	3
2. OBJETIVO DO PROJETO.....	4
2.1. Objetivos Específicos.....	4
3. COMPONENTES UTILIZADOS.....	5
4. DESENVOLVIMENTO DO CIRCUITO.....	6
4.1. Lógica de Fases Operacionais.....	6
4.2. Thresholds de Decisão	6
5. CÓDIGO DESENVOLVIDO – PARTES PRINCIPAIS	7
5.1. Definição das Fases da Missão e Thresholds.....	7
5.2. Leitura dos Sensores (DHT22, LDR e MPU-6050)	8
5.3. Avaliação de Fase.....	9
5.4. Avaliação de Fase: Alerta, Eclipse e Nominal.....	10
5.5. Loop Principal e Controle Não-Bloqueante	11
5.6. Acionamento dos LEDs por Fase.....	12
6. RESULTADOS OBTIDOS	13
6.1. Cenário Testado 1 - Todos os sensores em faixa normal	14
6.2. Cenário Testado 2 - LDR > 3000 (escuro)	15
6.3. Cenário Testado 3 - Temperatura > 60°C	16
6.4. Cenário Testado 4 - Umidade < 40%.....	17
6.5. Cenário Testado 5 - Vibração > 2,5g	18
6.6. Cenário Testado 6 - Temperatura < -5°C.....	19
6.7. Cenário Testado 7 - Vibração > 3,4g	20
6.8. Cenário Testado 8 - Umidade > 90%.....	21
7. MELHORIAS FUTURAS.....	22
8. CONCLUSÃO	23
9. ENTREGÁVEIS (LINKS)	24

1. INTRODUÇÃO

A nova corrida espacial não é movida apenas por foguetes, é movida por código. Dados coletados em órbita já orientam o agronegócio, antecipam desastres climáticos, monitoram desmatamento e conectam regiões remotas. A economia espacial global superou US\$ 630 bilhões em 2023 e projeta-se ultrapassar US\$ 1 trilhão até 2030, impulsionada por serviços digitais e sensoriamento orbital.

O Projeto E.D.E.N - Ecological Development in Exo-Environments, parte de uma hipótese científica: a exposição controlada de espécimes vegetais da Mata Atlântica à microgravidade e à radiação cósmica podem induzir adaptações fisiológicas que aumentem sua resiliência a condições climáticas extremas. Experimentos conduzidos pela NASA e pela ESA com sementes em órbita já demonstraram que o ambiente espacial provoca alterações genéticas e metabólicas mensuráveis.

Para que esse experimento seja viável, as condições internas da biocápsula precisam ser mantidas dentro de faixas precisas durante toda a missão. É aí que o sistema IoT desenvolvido neste projeto atua: monitorando continuamente temperatura, umidade, luminosidade e vibração, classificando o estado operacional da cápsula e acionando respostas físicas proporcionais à gravidade dos eventos detectados. O projeto conecta-se aos ODS 9 (Inovação), 13 (Ação Climática) e 15 (Vida Terrestre) da Agenda 2030 da ONU.

2. OBJETIVO DO PROJETO

Desenvolver um sistema embarcado IoT funcional no Wokwi com ESP32, capaz de coletar, processar e exibir em tempo real as variáveis físicas críticas da biocápsula E.D.E.N, temperatura, umidade, luminosidade e aceleração, classificando automaticamente o estado da missão em quatro fases operacionais e acionando atuadores conforme a gravidade detectada.

2.1. Objetivos Específicos

- Coletar temperatura e umidade via DHT22 (GPIO 4) e luminosidade via LDR/ADC (GPIO 34);
 - Medir vibração/aceleração nos 3 eixos via MPU-6050 (I2C: SDA=21, SCL=22), calculando a magnitude em g;
 - Exibir dados e alertas em LCD 16x2 I2C, com rotação automática de telas no modo nominal;
 - Implementar FSM com 4 fases (Nominal, Eclipse, Alerta, Evacuação) baseada em thresholds por variável;
 - Acionar LEDs indicadores e buzzer com padrao sonoro diferenciado por gravidade do evento;
 - Validar o sistema com testes sistemáticos cobrindo todos os cenários de fase
-

3. COMPONENTES UTILIZADOS

O sistema foi projetado em torno do ESP32 DevKit V1, escolhido por seu ADC de 12 bits (4x a resolução do Arduino Uno), suporte nativo a I2C fast mode, Wi-Fi integrado para evoluções futuras e GPIOs suficientes para todos os atuadores sem multiplexação. A tabela abaixo detalha cada componente e sua função no circuito:

Componente	Função	Interface / Pino
ESP32	Microcontrolador principal	-
DHT22	Temperatura e umidade	Digital - GPIO 4
LDR (fotoresistor)	Luminosidade ambiental	Analógico - GPIO 34
MPU-6050	Aceleração / vibração (3 eixos)	I ² C - SDA 21 / SCL 22
LCD 16×2 I ² C	Display de dados e alertas	I ² C - addr 0x27 (SDA 21 / SCL 22)
LED Verde	Indicador: Missão Nominal	GPIO 25
LED Amarelo	Indicador: Eclipse / Alerta	GPIO 26
LED Vermelho	Indicador: Evacuação	GPIO 27
LED Azul	Propulsor piscante (evacuação)	GPIO 32
Buzzer	Alertas sonoros	GPIO 23
Resistores (220Ω)	Limitadores de corrente dos LEDs	Em série com cada LED

Tabela 1 — Componentes do sistema E.D.E.N e suas funções.

4. DESENVOLVIMENTO DO CIRCUITO

O ESP32 atua como hub central, coletando dados dos sensores a cada 2 segundos e processando a lógica de fases de forma cíclica. O barramento I²C é compartilhado pelo MPU-6050 e pelo LCD, utilizando endereços distintos (0x68 e 0x27, respectivamente). O DHT22 opera em protocolo one-wire proprietário no GPIO 4, enquanto o LDR é lido via ADC de 12 bits no GPIO 34.

4.1. Lógica de Fases Operacionais

O sistema classifica continuamente o estado da missão em quatro fases, em ordem crescente de prioridade:

■ MISSÃO NOMINAL	Todas as variáveis dentro dos limites seguros. LED verde aceso, demais LEDs apagados, buzzer silencioso. O LCD alterna entre telas mostrando temperatura, umidade, luminosidade e aceleração.
■ ZONA DE ECLIPSE	LDR detecta ausência de luz (valor ADC > 3000). Indica que a cápsula entrou na sombra da Terra. LED amarelo aceso. Mensagem: ECLIPSE: REPOUSO. Sistema aguarda a saída do eclipse.
■ ■ ALERTA DA CÁPSULA	Uma ou mais variáveis cruzam os limiares de estresse (não críticos). Um único bip é emitido. LED amarelo + vermelho acesos. Mensagem específica por causa indica a ação corretiva em curso.
■ EVACUANDO!!!	Variáveis em nível crítico — risco estrutural ou de colapso do suporte à vida. Buzzer contínuo, LED azul piscando simulando o acionamento dos propulsores de reentrada. Mensagem de falha específica na linha 1, REENTRADA INIC. ou EVACUANDO! na linha 2.

4.2. Thresholds de Decisão

Variável	Estresse (Alerta)	Crítico (Evacuação)
Temperatura	< 0 °C ou > 60 °C	< -5 °C ou > 80 °C
Umidade	< 40 % ou > 85 %	< 10 % ou > 90 %
Vibração	> 2,5 g	> 3,4 g
Luminosidade (LDR ADC)	> 3.800 (estresse)	> 4.000 (crítico) / > 3.000 (eclipse)

Tabela 2 — Thresholds para transição de fases operacionais.

5. CÓDIGO DESENVOLVIDO – PARTES PRINCIPAIS

O firmware foi desenvolvido em C++ para a plataforma Arduino/ESP32. A seguir são apresentados os trechos principais com comentários explicativos. O código completo está disponível no repositório GitHub do projeto.

5.1. Definição das Fases da Missão e Thresholds

```
// — FASES DA MISSÃO —
enum MissionPhase {
    PHASE_NOMINAL,    // Missão nominal – condições ideais de suporte à vida
    PHASE_ECLIPSE,    // Zona de eclipse – aguardando passagem
    PHASE_STRESS,     // Alerta – sistema corrigindo condições da cápsula
    PHASE_REENTRY     // Crítico – sistemas falharam, evacuação necessária
};

MissionPhase currentPhase = PHASE_NOMINAL;
String phaseNames[] = {"MISSAO NOMINAL", "ZONA ECLIPSE", "ALERTA CAPSULA", "EVACUANDO!!!"};

// — THRESHOLDS POR FASE —
// [NOMINAL, ECLIPSE, STRESS, REENTRY]
float tempMax[] = {35.0, 25.0, 60.0, 80.0}; // °C – limite superior
float tempMin[] = {10.0, 0.0, 0.0, -5.0}; // °C – limite inferior (estresse < 0, reentrada < -5)
float humMax[] = {70.0, 75.0, 85.0, 90.0}; // % – umidade máxima (acima = risco de fungos)
float humMin[] = {40.0, 35.0, 40.0, 10.0}; // % – umidade mínima (estresse < 40%, reentrada < 10%)
// LDR no Wokwi: valor BAIXO = muita luz | valor ALTO = escuro
// Range real: ~0 (luz máxima) a ~4095 (escuridão total)
// Eclipse só ocorre com luz bastante reduzida
int ldrMax[] = {1500, 3000, 3800, 4000}; // acima desse valor = escuro demais p/ a fase
float vibrMax[] = {1.5, 1.0, 2.5, 3.4}; // g – aceleração máxima (Wokwi max ~3.46g)
```

Define os 4 estados possíveis da biocápsula (Nominal, Eclipse, Stress e Reentrada) usando um enum, e estabelece os limites aceitáveis de temperatura, umidade, luminosidade e vibração para cada fase. É a base lógica de todo o sistema.

5.2. Leitura dos Sensores (DHT22, LDR e MPU-6050)

```
// — LÊ SENSORES —  
void readSensors() {  
    // DHT22  
    float t = dht.readTemperature();  
    float h = dht.readHumidity();  
    if (!isnan(t)) temperature = t;  
    if (!isnan(h)) humidity = h;  
  
    // LDR  
    ldrValue = analogRead(LDR_PIN);  
  
    // MPU-6050 — calcula magnitude da aceleração  
    int16_t ax, ay, az, gx, gy, gz;  
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);  
    float ax_g = ax / 16384.0;  
    float ay_g = ay / 16384.0;  
    float az_g = az / 16384.0;  
    vibration = sqrt(ax_g*ax_g + ay_g*ay_g + az_g*az_g);  
}
```

readSensors() Responsável pela leitura dos três sensores: o DHT22 fornece temperatura e umidade, o LDR retorna o nível de luminosidade via analogRead, e o MPU-6050 fornece os dados de aceleração. A magnitude da vibração é calculada pela fórmula vetorial $\sqrt{ax^2 + ay^2 + az^2}$, convertendo os dados brutos em uma unidade em "g".

5.3. Avaliação de Fase

```
// — AVALIA FASE DA MISSÃO —
void evaluatePhase() {
    alertActive = false;

    bool tempCritical = (temperature > tempMax[PHASE_REENTRY]) ||
    || (temperature < tempMin[PHASE_REENTRY]);
    bool vibrCritical = (vibration > vibrMax[PHASE_REENTRY]);
    bool humCritical = (humidity > humMax[PHASE_REENTRY]) ||
    || (humidity < humMin[PHASE_REENTRY]);
    bool ldrCritical = (ldrValue > ldrMax[PHASE_REENTRY]);

    // EVACUAÇÃO: atuadores falharam – condições letais para a planta
    if (tempCritical || vibrCritical || humCritical || ldrCritical) {
        if (tempCritical) {
            alertCause = temperature > tempMax[PHASE_REENTRY] ? "FALHA TERMICA" : "CONGELAMENT
        } else if (vibrCritical) {
            alertCause = "DANO ESTRUTURAL";
        } else if (humCritical) {
            alertCause = "COLAPSO HIDRICO";
        } else {
            alertCause = "FALHA ENERGETICA";
        }
        if (currentPhase != PHASE_REENTRY) {
            currentPhase = PHASE_REENTRY;
            alertActive = true;
            reentryAlert();
        }
        return;
    }
}
```

evaluatePhase() (parte 1 - Reentrada) Verifica se alguma variável atingiu níveis críticos irreversíveis. Caso sim, identifica a causa (falha térmica, dano estrutural, colapso hídrico ou falha energética) e aciona o protocolo de reentrada.

5.4. Avaliação de Fase: Alerta, Eclipse e Nominal

```
// ALERTA: sistema de suporte falhou – IA ativa atuadores para corrigir
bool tempHigh = temperature > tempMax[PHASE_STRESS];
bool tempLow  = temperature < tempMin[PHASE_STRESS];
bool vibrHigh = vibration   > vibrMax[PHASE_STRESS];
bool humHigh  = humidity    > humMax[PHASE_STRESS];
bool humLow   = humidity    < humMin[PHASE_STRESS];
bool ldrHigh  = ldrValue    > ldrMax[PHASE_STRESS];

bool inStress = tempHigh || tempLow || vibrHigh || humHigh || humLow || ldrHigh;

if (inStress) {
    // Define mensagem do atuador correspondente
    if (tempHigh)    alertCause = "TEMP+:RESFRIANDO";
    else if (tempLow) alertCause = "TEMP-: AQUECENDO";
    else if (humHigh) alertCause = "UMID+:EXAUSTORES";
    else if (humLow)  alertCause = "UMID-: IRRIGANDO";
    else if (vibrHigh) alertCause = "VIBR: ESTABILIZ.";
    else if (ldrHigh) alertCause = "ECLIPSE: REPOUSO";
    // Reseta bip se a causa mudou
    if (alertCause != lastAlertCause) {
        stressBeepDone = false;
        lastAlertCause = alertCause;
    }
    currentPhase = PHASE_STRESS;
    alertActive  = true;
    return;
}

// ECLIPSE: luz reduzida – aguardando passagem
if (ldrValue > ldrMax[PHASE_ECLIPSE]) {
    alertCause    = "ECLIPSE ORBITAL";
    stressBeepDone = false;
    lastAlertCause = "";
    currentPhase  = PHASE_ECLIPSE;
    return;
}

// NOMINAL: tudo dentro do esperado
alertCause    = "";
stressBeepDone = false;
lastAlertCause = "";
currentPhase  = PHASE_NOMINAL;
```

evaluatePhase() (parte 2 - Stress, Eclipse e Nominal) Continua a avaliação para condições menos críticas: se algum sensor estiver fora do ideal, o sistema entra em Stress e registra qual atuador foi acionado para corrigir. Se apenas a luz estiver baixa, entra em Eclipse. Caso tudo esteja dentro do esperado, retorna ao estado Nominal.

5.5. Loop Principal e Controle Não-Bloqueante

```
void loop() {
    unsigned long now = millis();

    // Lê sensores a cada 2s
    if (now - lastPhaseCheck >= 2000) {
        lastPhaseCheck = now;
        readingCount++;

        readSensors();
        evaluatePhase();
        updateLEDs();
        logToSerial();
    }

    // Alterna página do LCD a cada 3s
    if (now - lastPageSwitch >= 3000) {
        lastPageSwitch = now;
        displayPage = (displayPage + 1) % 3;
        updateLCD();
    }

    // Buzzer: 1 bip único no estresse, contínuo na reentrada
    if (currentPhase == PHASE_REENTRY) {
        // Bip contínuo na reentrada
        if (now - lastBuzzerBeep >= 400) {
            lastBuzzerBeep = now;
            tone(BUZZER_PIN, 1500, 300);
        }
        // LED azul piscando – propulsor ativado
        if (now - lastThruster >= 300) {
            lastThruster = now;
            thrusterState = !thrusterState;
            digitalWrite(LED_BLUE, thrusterState);
        }
    } else {
        // Fora da reentrada – propulsor desligado
        digitalWrite(LED_BLUE, LOW);
        thrusterState = false;
        if (currentPhase == PHASE_STRESS && !stressBeepDone) {
            // 1 bip único ao entrar no estresse
            tone(BUZZER_PIN, 800, 200);
            stressBeepDone = true;
        }
    }
}
```

loop () Orquestra todas as funções do sistema usando millis() no lugar de delay(), permitindo que leitura de sensores, atualização do LCD, controle do buzzer e pisca do LED azul ocorram de forma simultânea e não-bloqueante.

5.6. Acionamento dos LEDs por Fase

```
// — ATUALIZA LEDs —  
void updateLEDs() {  
    digitalWrite(LED_GREEN, LOW);  
    digitalWrite(LED_YELLOW, LOW);  
    digitalWrite(LED_RED, LOW);  
  
    switch (currentPhase) {  
        case PHASE_NOMINAL:  
            digitalWrite(LED_GREEN, HIGH);  
            break;  
        case PHASE_ECLIPSE:  
            digitalWrite(LED_YELLOW, HIGH);  
            break;  
        case PHASE_STRESS:  
            digitalWrite(LED_YELLOW, HIGH);  
            digitalWrite(LED_RED, HIGH);  
            break;  
        case PHASE_REENTRY:  
            digitalWrite(LED_RED, HIGH);  
            break;  
    }  
}
```

updateLEDs() Traduz a fase atual em sinal visual: verde para Nominal, amarelo para Eclipse, amarelo e vermelho para Stress, e vermelho para Reentrada. O LED azul, que representa o propulsor, é controlado separadamente no loop().

6. RESULTADOS OBTIDOS

A simulação no Wokwi foi executada com sucesso, validando todos os cenários de missão previstos. Os testes foram conduzidos manipulando manualmente os valores dos sensores simulados para forçar transições entre cada fase operacional.

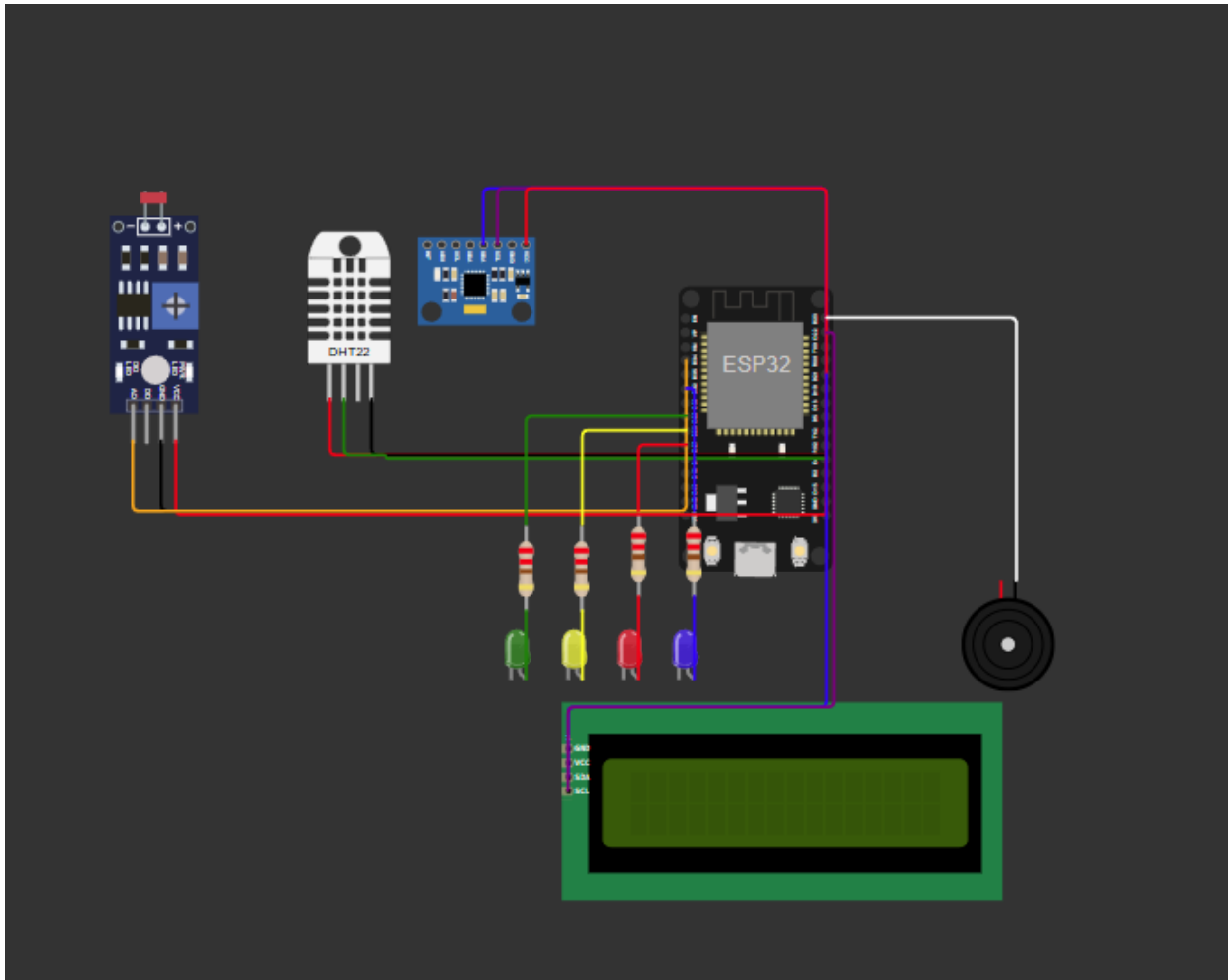
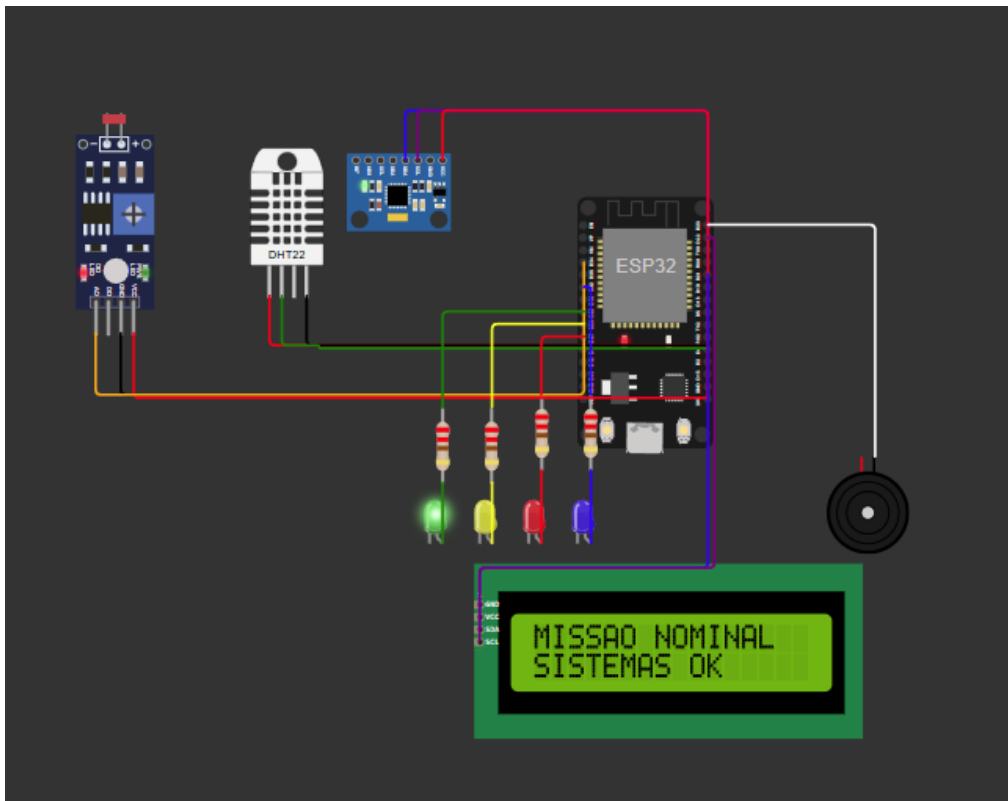


Imagem do circuito finalizado

6.1. Cenário Testado 1 - Todos os sensores em faixa normal

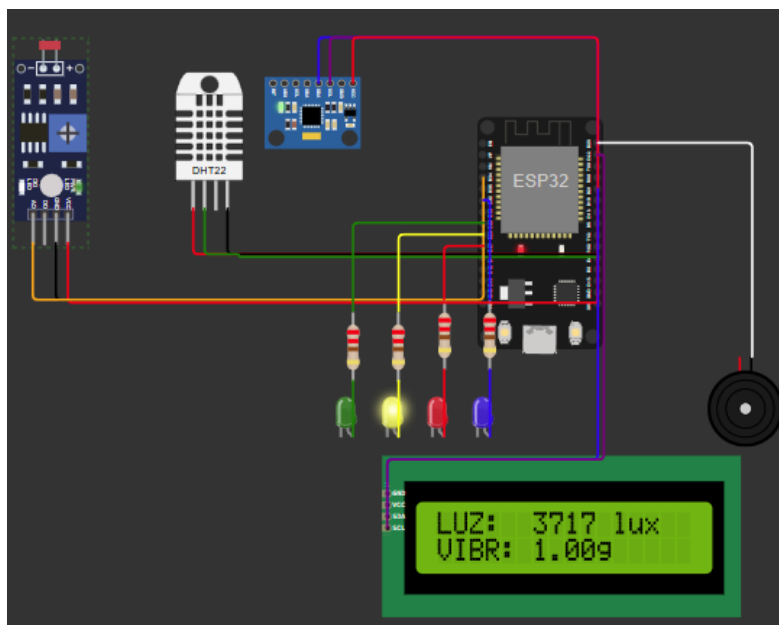
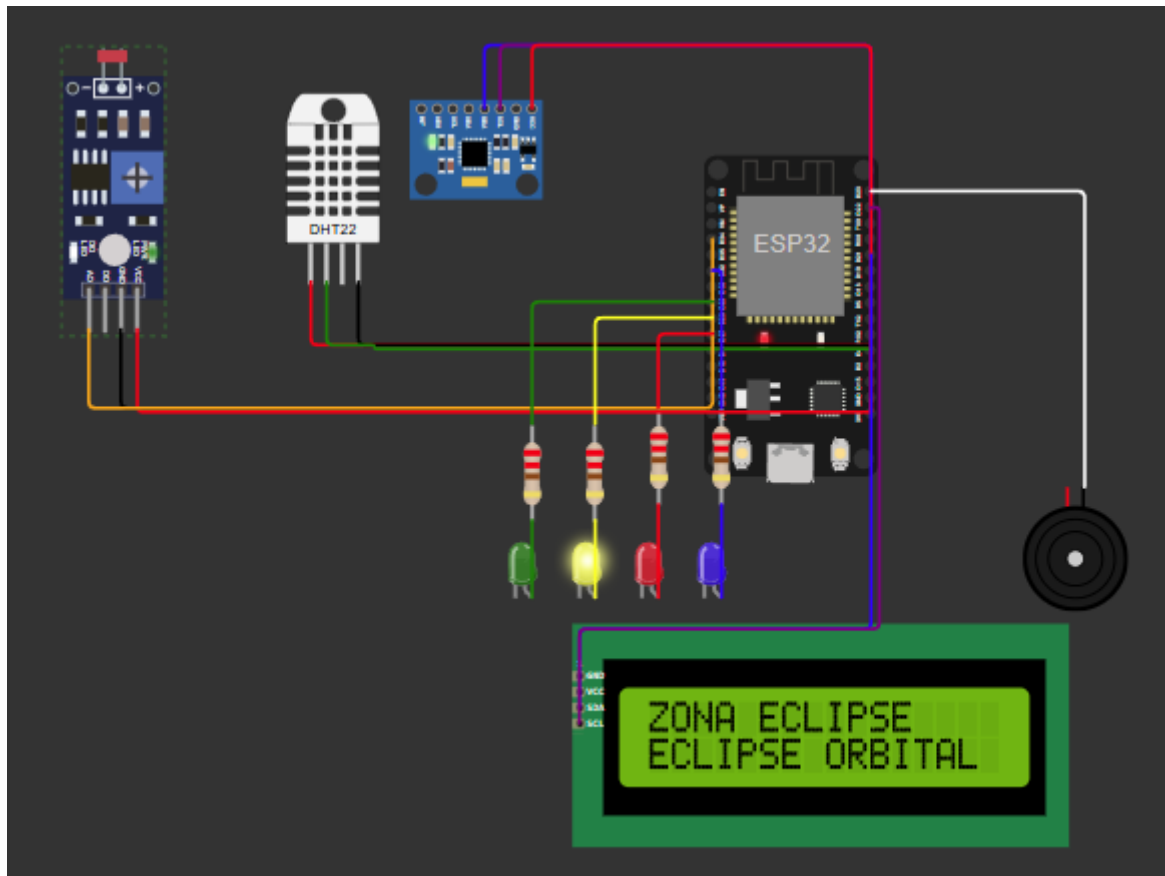


RESULTADO ESPERADO: MISSÃO NOMINAL (LED VERDE)

RESULTADO OBTIDO: LED VERDE ACESO, LCD ROTACIONANDO TELAS

STATUS: OK

6.2. Cenário Testado 2 - LDR > 3000 (escuro)

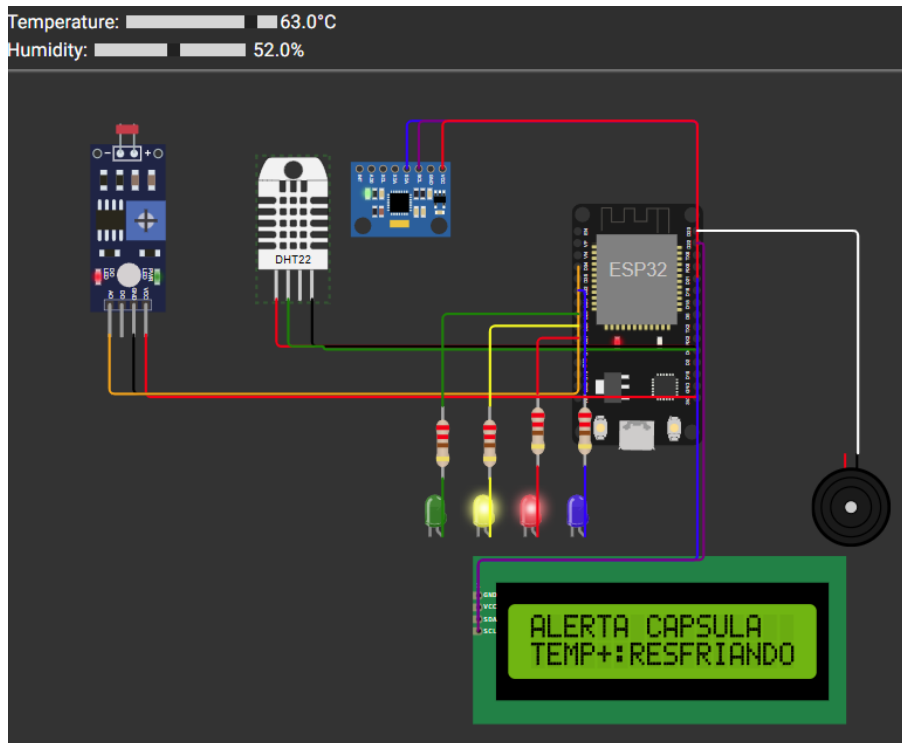


RESULTADO ESPERADO: ZONA DE ECLIPSE (LED AMARELO)

RESULTADO OBTIDO: LED amarelo, mensagem ECLIPSE: REPOUSO no LCD.

STATUS: OK

6.3. Cenário Testado 3 - Temperatura > 60°C

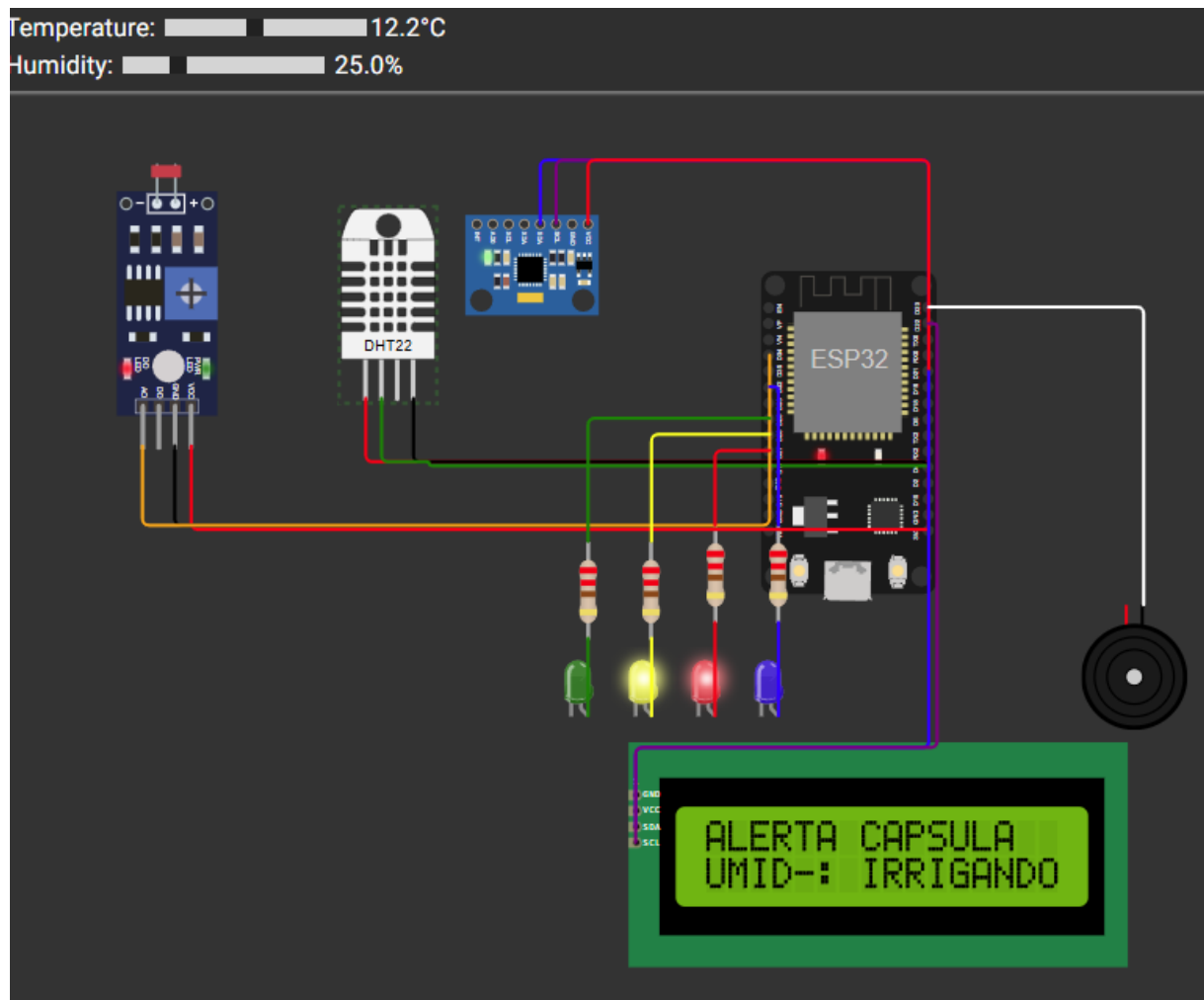


RESULTADO ESPERADO: ALERTA CÁPSULA 1 BIP, LED AMARELO + VERMELHO

RESULTADO OBTIDO: BIP ÚNICO, LEDS AMARELO + VERMELHO, TEMP+:RESFRIANDO

STATUS: OK

6.4. Cenário Testado 4 - Umidade < 40%

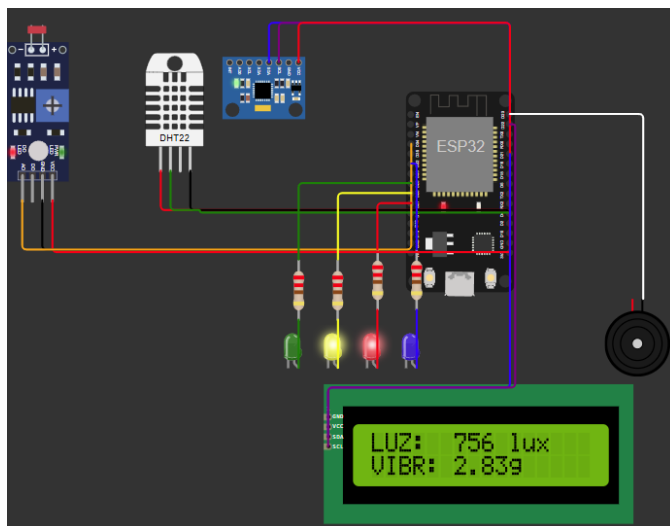
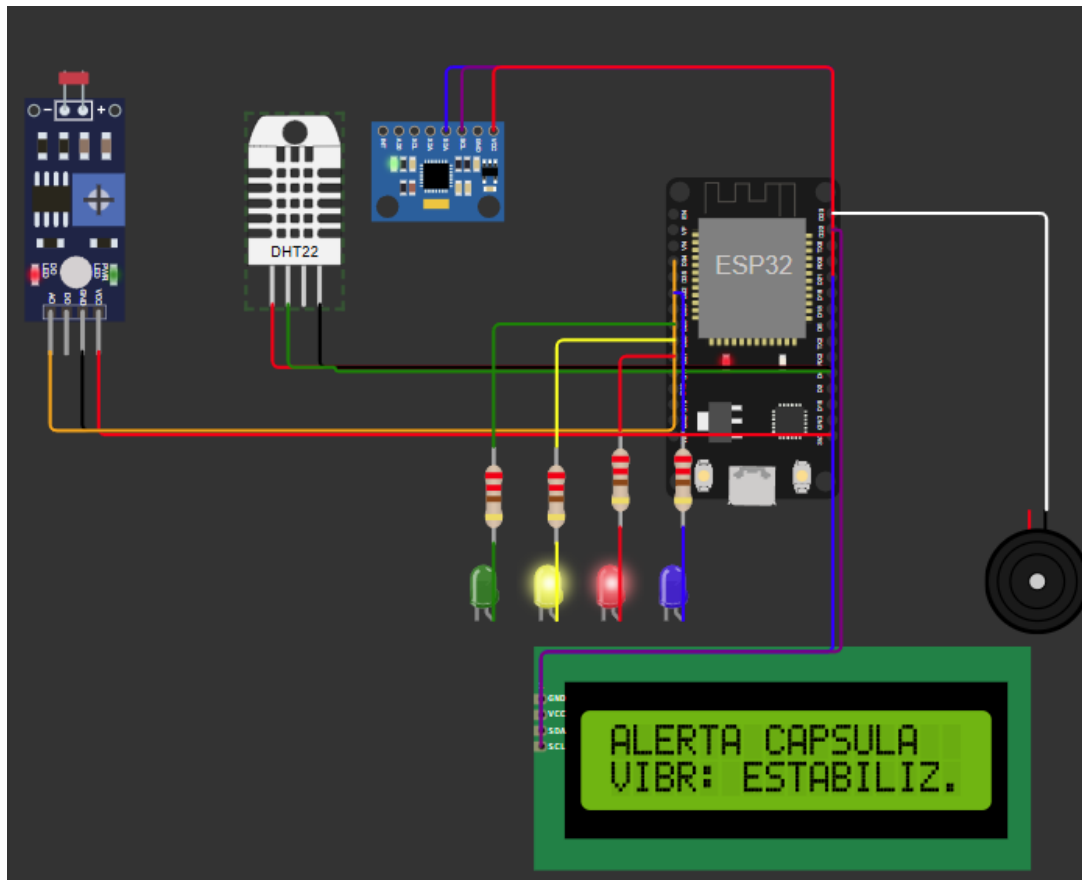


RESULTADO ESPERADO: ALERTA CÁPSULA UMID-: IRRIGANDO

RESULTADO OBTIDO: MENSAGEM UMID-: IRRIGANDO, BIP, LEDS AMARELO + VERMELHO

STATUS: OK

6.5. Cenário Testado 5 - Vibração > 2,5g

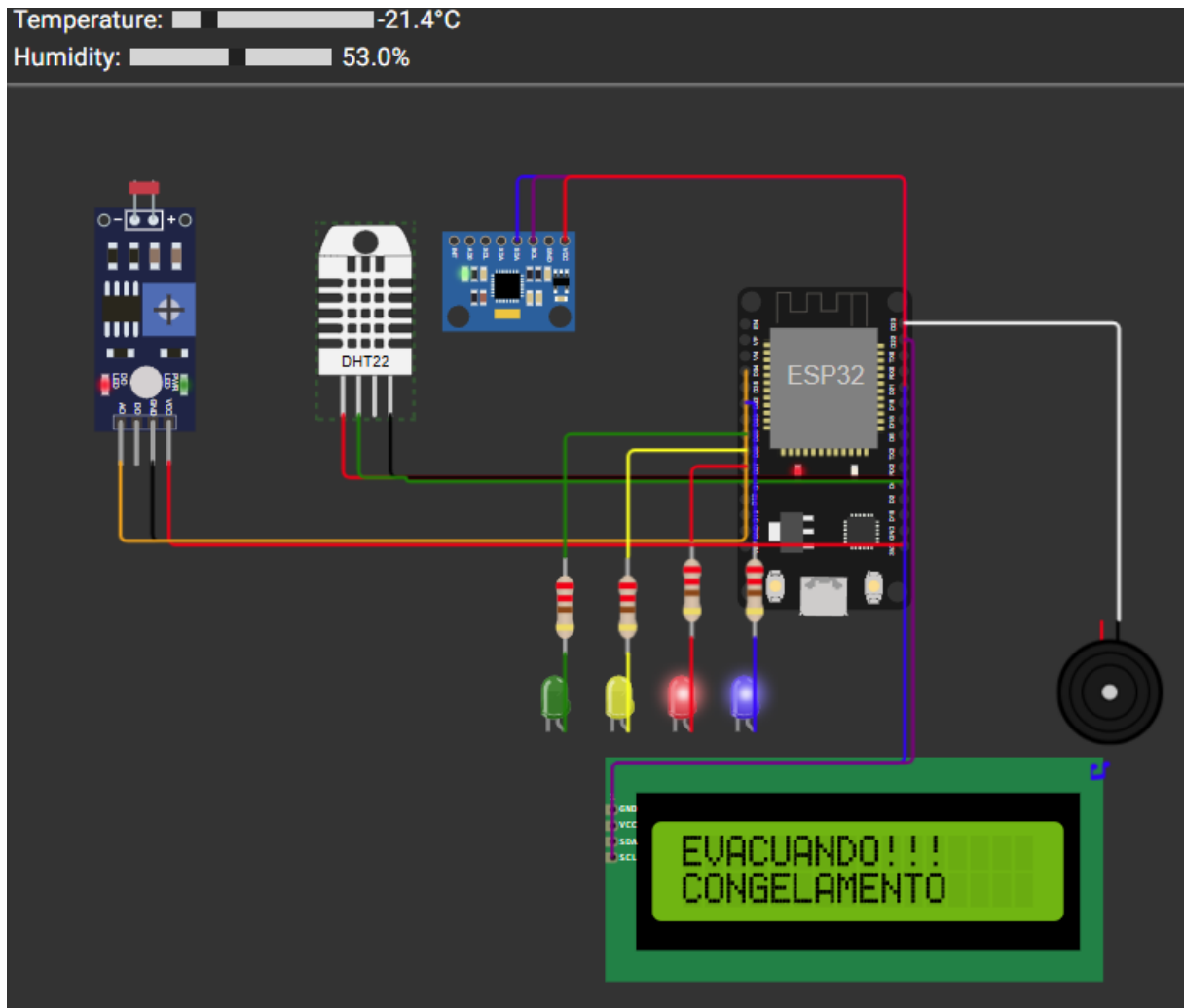


RESULTADO ESPERADO: ALERTA CÁPSULA VIBR: ESTABILIZ

RESULTADO OBTIDO: MENSAGEM VIBR: ESTABILIZ. EXIBIDA CORRETAMENTE

STATUS: OK

6.6. Cenário Testado 6 - Temperatura < -5°C

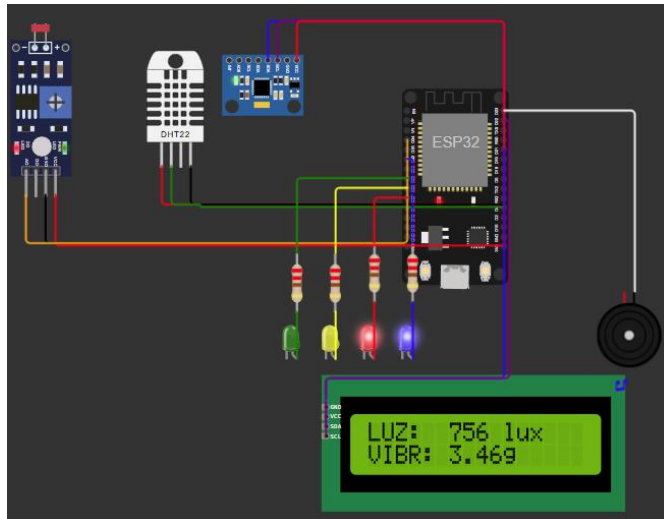
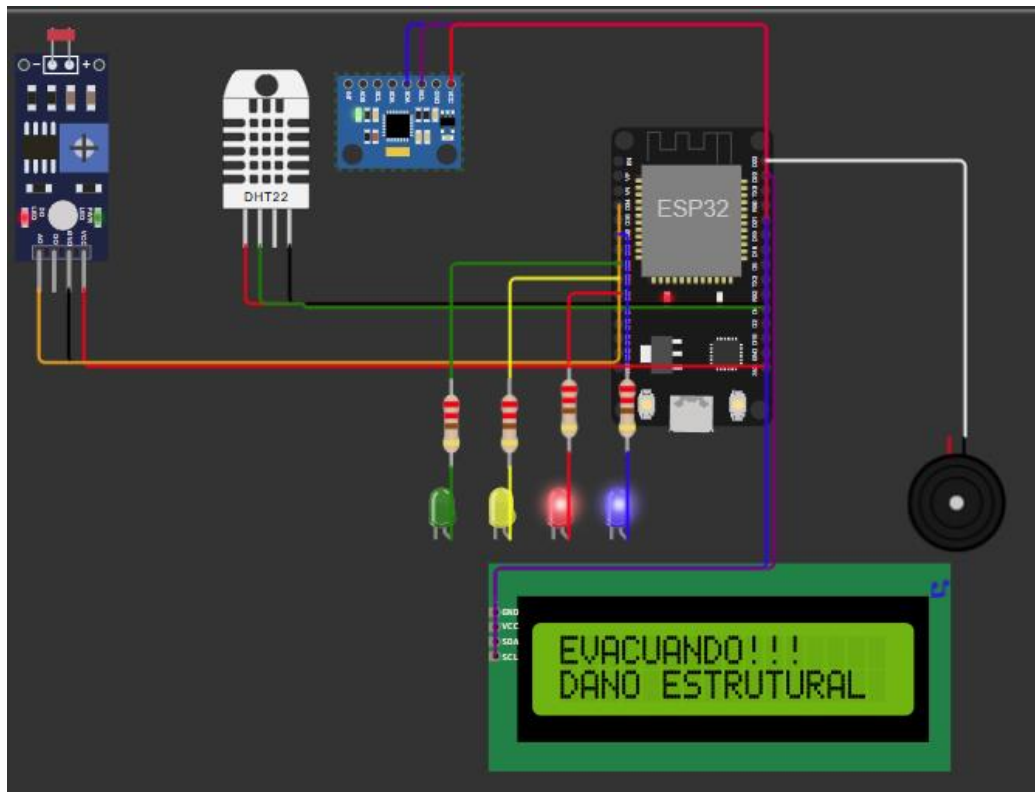


RESULTADO ESPERADO: EVACUANDO BUZZER CONTÍNUO

RESULTADO OBTIDO: BUZZER CONTÍNUO, LED AZUL PISCANDO, FALHA TERMICA REENTRADA INIC.

STATUS: OK

6.7. Cenário Testado 7 - Vibração > 3,4g

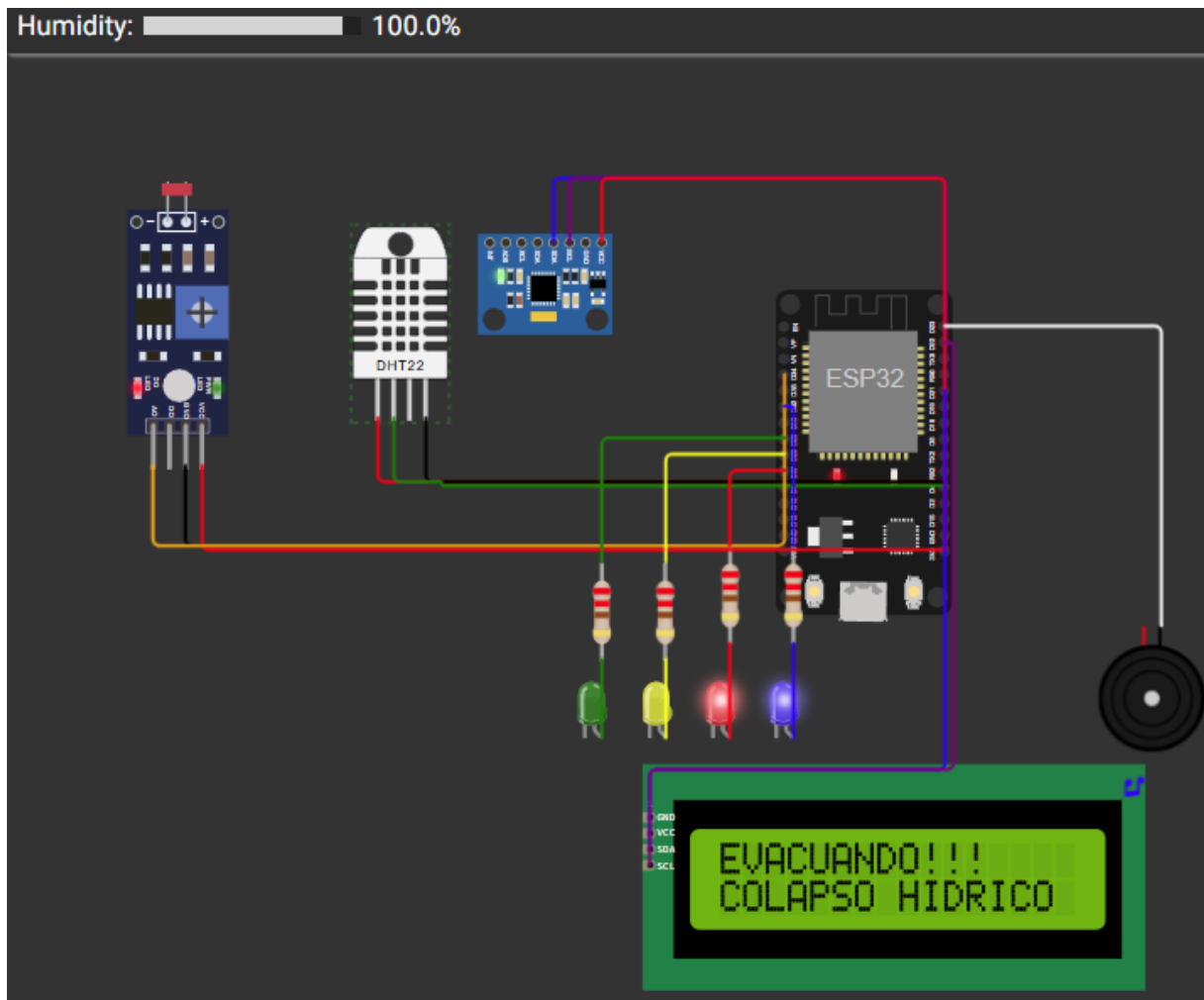


RESULTADO ESPERADO: EVACUANDO DANO ESTRUTURAL

RESULTADO OBTIDO: FASE EVACUAÇÃO, MENSAGEM DANO ESTRUTURAL NO LCD

STATUS: OK

6.8. Cenário Testado 8 - Umidade > 90%



RESULTADO ESPERADO: EVACUANDO COLAPSO HIDRICO

RESULTADO OBTIDO: EVACUAÇÃO INICIADA, MENSAGEM COLAPSO HIDRICO

STATUS: OK

O sistema respondeu corretamente a todos os 8 cenários testados, incluindo transições simultâneas de múltiplas variáveis críticas. A prioridade entre fases foi respeitada em todos os casos: quando temperatura e vibração atingiram níveis críticos simultaneamente, o sistema manteve a fase de evacuação e exibiu a causa de maior gravidade.

7. MELHORIAS FUTURAS

Transmissão via MQTT/LoRa	Envio dos dados da cápsula para uma estação terrestre usando MQTT over Wi-Fi (ESP32) ou LoRa para comunicação de longo alcance sem infraestrutura.
Dashboard web em tempo real	Interface web com Node-RED ou Grafana exibindo histórico de leituras, gráficos de tendência e mapa de alertas — acessível via IP local do ESP32.
Sensor de CO ₂ (MQ-135)	Adição de sensor de dióxido de carbono para monitorar a fotossíntese das plantas e detectar anomalias metabólicas nos espécimes botânicos.
Sensor de pressão atmosférica (BMP280)	Monitoramento da pressão interna da cápsula para detecção precoce de micro-fraturas estruturais — variável crítica em ambientes de vácuo externo.
Machine Learning embarcado	TinyML no ESP32 para predição de falhas com base em padrões históricos de vibração, antecipando eventos críticos antes que os thresholds sejam atingidos.
Hardware físico com PCB dedicada	Migração da simulação Wokwi para prototipagem física com PCB customizada, caixa impressa em 3D e testes em câmara de temperatura.

8. CONCLUSÃO

O Projeto E.D.E.N demonstrou que é viável desenvolver um sistema IoT completo e funcional para monitoramento de ambientes espaciais críticos utilizando hardware acessível e plataformas de simulação como o Wokwi. A integração entre sensores heterogêneos (DHT22, LDR, MPU-6050), display I²C, atuadores físicos (LEDs, buzzer) e uma máquina de estados robusta resultou em um sistema coeso e responsivo.

A escolha do ESP32 como microcontrolador mostrou-se acertada: sua capacidade de processamento, múltiplas interfaces (I²C, ADC, GPIO) e suporte nativo a Wi-Fi abrem caminho direto para a evolução do projeto em direção à transmissão de dados em tempo real, próximo passo natural da solução.

Do ponto de vista educacional, o desafio exigiu a integração simultânea de conceitos de sistemas embarcados, programação orientada a eventos, eletrônica analógica e digital, e design de experiência de usuário (clareza das mensagens no LCD de apenas 16 caracteres). A restrição de espaço no display foi, ironicamente, um dos maiores desafios de engenharia, cada mensagem precisava ser ao mesmo tempo informativa, precisa e legível em uma situação de emergência simulada.

O conceito central, preservar espécies da Mata Atlântica através da exposição controlada ao espaço para desenvolver resiliência genética, conecta ciência espacial à preservação ambiental de forma genuinamente inovadora, alinhando-se com os ODS 13 e 15 da Agenda 2030 da ONU.

9. ENTREGÁVEIS (LINKS)

- **LINK DO PROJETO NO WOKWI:**

<https://wokwi.com/projects/465840600415942657>

- **LINK DO VÍDEO NO YOUTUBE:**

<https://youtu.be/-6Ov3c38Vsc>

- **LINK DO GITHUB:**

<https://github.com/ExoGenesis-PROJETO-E-D-E-N/COA-PROJETO-EDEN-PROTOTIPO/tree/main>